Lab 10 — CSV và phân tích điểm sinh viên (lời giải từ PythoClass/main.py)

1. Đề bài tiếng Anh

LAB Session 10

Objective

Develop a Python program to manage student grades using **File I/O, Functions, and Error Handling**.

Requirements

Create a menu-driven program with these functions:

1. **Display All Students:** Read/display student data from `students.csv`.
2. **Add a New Student:** Enter student information and grades. Validate input and prevent duplicate Student IDs.
3. **Search Student by ID:** Search/display a student's information.
4. **Calculate Average Grade:** Display each student's average grade.
5. **Save Students to File:** Save current student data to `students.csv`.
6. **Exit:** Exit the program.

Error Handling

Handle file-not-found errors, invalid numeric input, grades outside **0–100**, and duplicate student IDs.

Sample `students.csv`

```
student_id,name,class_name,Python,Math,ML
S001,Learner 01,AI01,85,78,90
S002,Learner 02,AI01,72,85,80
S003,Learner 03,AI01,90,88,95
S004,Learner 04,AI02,65,70,68
S005,Learner 05,AI02,82,75,85
S006,Learner 06,AI02,78,80,72
S007,Learner 07,AI03,95,92,94
S008,Learner 08,AI03,70,68,75
S009,Learner 09,AI03,88,85,82
S010,Learner 10,AI03,76,90,88
```

Chuyển từ `Lab Session 10.docx` sang Markdown. Tên học viên trong dữ liệu ví dụ đã được thay bằng mã minh họa nhằm tránh công khai thông tin định danh; điểm số, lớp và cấu trúc bảng được giữ nguyên.

2. Đề bài tiếng Việt

LAB SESSION 10 — Quản lý điểm sinh viên bằng file CSV (tệp phân cách bằng dấu phẩy)

Mục tiêu

Xây dựng chương trình Python quản lý điểm sinh viên bằng thao tác **File I/O**, các **hàm** và **xử lý lỗi**.

Các yêu cầu

Tạo chương trình có menu với sáu chức năng:

1. **Hiển thị tất cả sinh viên:** Đọc dữ liệu sinh viên từ `students.csv` và hiển thị thông tin.
2. **Thêm sinh viên:** Nhập thông tin cá nhân và điểm của từng môn; kiểm tra dữ liệu hợp lệ và chống mã sinh viên trùng.
3. **Tìm theo mã sinh viên:** Nhập mã và hiển thị thông tin của sinh viên cần tìm.
4. **Tính điểm trung bình:** In điểm trung bình từng sinh viên.
5. **Lưu vào file:** Ghi dữ liệu đang quản lý vào `students.csv`.
6. **Thoát:** Kết thúc chương trình.

Xử lý lỗi

Cần xử lý **không tìm thấy file**, **nhập số không hợp lệ**, **điểm ngoài phạm vi 0–100** và **mã sinh viên trùng**.

Dữ liệu mẫu students.csv (tệp phân cách bằng dấu phẩy)

Dùng bảng CSV gồm mã sinh viên, tên, lớp, điểm Python, Math và ML. Bảng được giữ nguyên số liệu, tên đã được thay bằng mã/tên giả lập để bảo vệ danh tính:

```
student_id,name,class_name,Python,Math,ML
S001,Learner 01,AI01,85,78,90
S002,Learner 02,AI01,72,85,80
S003,Learner 03,AI01,90,88,95
S004,Learner 04,AI02,65,70,68
S005,Learner 05,AI02,82,75,85
S006,Learner 06,AI02,78,80,72
S007,Learner 07,AI03,95,92,94
S008,Learner 08,AI03,70,68,75
S009,Learner 09,AI03,88,85,82
S010,Learner 10,AI03,76,90,88
```

Các cột của file là `student_id,name,class_name,Python,Math,ML`; tên `Learner 01..10` là dữ liệu minh họa đã được ẩn danh.

3. Code Python có chú thích

```
import pandas as pd  # Pandas (thư viện xử lý bảng dữ liệu) với tên ngắn pd.

def inputHelper(data):  # Đọc đầu vào; chú ý kiểm tra kiểu bên dưới chưa chuyển chuỗi sang số.
    label = data["label"]
    formats = data["formats"]
    inputTemp = input(label)
    if inputTemp == "":
        return(None)
    blocks =  inputTemp.split(";")  # Tách chuỗi đầu vào tại dấu chấm phẩy.


    if len(blocks) != len(formats):
        print("!!![ERROR] Input blocks is not correct!~")


    for i in range(len(formats)):
        if type(blocks[i]).__name__ != formats[i]:  # LƯU Ý: split trả chuỗi nên so kiểu bằng tên không thực sự xác thực dữ liệu.
```

```
            print("!!![ERROR] Input format is not corrects")
    return(blocks)

def displayAll():  # In danh sách tên theo cột có khoảng trắng đầu trong CSV gốc.
    df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
    print("""
==== DISPLAY ALL STUDENT =====""")
    students = df[" name"].values  # Lấy cột ' name' có dấu cách đầu theo CSV gốc PythoClass.
    stt = 1
    for student in students:
        print(f"{stt}. {student}")
        stt += 1

def studentSearch():  # Lọc theo mã sinh viên.
    print("""
==== SEARCH STUDENT =====""")
    inputData = inputHelper({
        "label":"Please enter Student ID:",
        "formats":["str"]
    })
    if inputData:
        inputData = inputData[0]
        df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
        result = df[df["student_id"] == inputData]
        if len(result)!=0:
            print(result)
        else:
            print("!!![ERROR] Student not found.")
        studentSearch()  # Gọi lại chính hàm tìm kiếm; lặp nhiều có thể tạo đệ quy quá sâu.
    else:
        return

def studentAverage():  # Tính trung bình ba môn cho một sinh viên.
    print("""
==== STUDENT RESULT ====""")
    inputData = inputHelper({
        "label":"Please enter Student ID:",
        "formats":["str"]
    })
    if inputData:
        inputData = inputData[0]
        df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
        result = df[(df["student_id"] == inputData)]
        if len(result)!=0:
            values = result.values
            print(f"""
Student name: {values[0][1]}
Python score: {values[0][3]}
Math score: {values[0][4]}
ML score: {values[0][5]}
Average score:{round((values[0][3] + values[0][4] + values[0][5])/3,2)}
Highest score:{max(values[0][3],values[0][4],values[0][5])}
Lowest score:{min(values[0][3],values[0][4],values[0][5])}
""")
        else:
            print("!!![ERROR] Student not found.")
        studentAverage()  # Gọi lại hàm tính điểm; lặp nhiều có thể tạo đệ quy quá sâu.
    else:
```

```
        return

def analyzeSubject():  # Thống kê theo từng cột môn học.
    print("""
==== SUBJECT PERFORMANCE ====""")
    df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
    data = {
        "Average":[df["Python"].mean(),df["Math"].mean(),df["ML"].mean()],
        "Total":[df["Python"].sum(),df["Math"].sum(),df["ML"].sum()],
        "Highest":[df["Python"].max(),df["Math"].max(),df["ML"].max()],
        "Lowest":[df["Python"].min(),df["Math"].min(),df["ML"].min()],
    }
    dfShow = pd.DataFrame(data,index=["Python","Math","ML"])
    print(dfShow)


def analyzeClass():  # groupby/merge tổng hợp kết quả theo lớp.
    print("""
===== CLASS PERFORMANCE =====""")
    df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
    studentCount =  df.groupby("class_name")["student_id"].count()  # Nhóm theo lớp và đếm mã sinh viên.
    pythonMean =  df.groupby("class_name")["Python"].mean()
    mathMean =  df.groupby("class_name")["Math"].mean()
    mlMean =  df.groupby("class_name")["ML"].mean()
    result = pd.merge(studentCount,pythonMean,on="class_name",how="outer")  # Ghép các thống kê theo khóa class_name.
    result = pd.merge(result,mathMean,on="class_name",how="outer")  # Ghép các thống kê theo khóa class_name.
    result = pd.merge(result,mlMean,on="class_name",how="outer")  # Ghép các thống kê theo khóa class_name.
    result["Overall Average"] = (result["Python"] + result["Math"] +  result["ML"])/3  # Trung bình chung từ trung bình ba môn.


    result.reset_index(inplace=True)
    result.rename(columns={"student_id":"Students"}, inplace=True)
    result.rename(columns={"class_name":"Class"}, inplace=True)

    result = result.round(2)

    print(result)

def top3():  # Chức năng 6: xếp hạng ba sinh viên có điểm trung bình cao nhất.
    print("""
===== TOP 3 STUDENTS =====""")
    df = pd.read_csv("students.csv")  # Đọc students.csv ở thư mục chạy hiện tại thành DataFrame.
    df["Average"] =   (df["Python"] + df["Math"] +  df["ML"])/3
    dfSorted = df.sort_values(by="Average",ascending=False)
    dfSorted.drop(columns=["Python","Math","ML"], inplace=True)
    dfSorted.rename(columns={"class_name":"Class","student_id":"Student ID"}, inplace=True)
    dfSorted = dfSorted.round(2)

    print(dfSorted.head(3))  # Chỉ hiển thị ba hàng đứng đầu.

# MENU()
def menu():  # Bảy lựa chọn của main.py gốc, không phải sáu chức năng của đề Word.
    choice = 0
    while True:
        print("""
================================
 STUDENT PERFORMANCE ANALYSIS
--------------------------------------------------------
```

```
1. Display all students
2. Search for a student
3. Calculate student average
4. Analyze subject performance
5. Analyze class performance
6. Show top 3 students
7. Exit
        """)

        inputData =input("Please select (1-7): ")
        if inputData.isnumeric():
            choice = int(inputData)
            if choice == 1:
                displayAll()
            elif choice == 2:
                studentSearch()  # Gọi lại chính hàm tìm kiếm; lặp nhiều có thể tạo đệ quy quá sâu.
            elif choice == 3:
                studentAverage()  # Gọi lại hàm tính điểm; lặp nhiều có thể tạo đệ quy quá sâu.
            elif choice == 4:
                analyzeSubject()
            elif choice == 5:
                analyzeClass()
            elif choice == 6:
                top3()
            elif choice == 7:
                break
            else:
                print("!!![ERROR] Please enter from 1 to 7")
        else:
            print("!!![ERROR] Please enter from 1 to 7")

# goi ham
menu()
```

4. Giải thích nội dung và cách giải

Phần 3 chứa **một chương trình Python thống nhất**. Dưới đây là giải thích chi tiết cách từng nhóm hàm giải quyết yêu cầu, không lặp lại các đoạn code rời.

Tổng quan chương trình thực tế của `main.py`

Tệp này là **bài giải đang có trong repo PythoClass**, sử dụng `pandas` để đọc và phân tích `students.csv`. Chương trình in menu **7 lựa chọn**: hiển thị, tìm sinh viên, thống kê một sinh viên, thống kê theo môn, theo lớp, xếp hạng 3 sinh viên đầu và thoát. Trong khi đó, đề Word Lab 10 yêu cầu menu **6 lựa chọn**, có **thêm sinh viên** và **lưu file**. Đây là **hai phạm vi không hoàn toàn giống nhau**, nên không được coi `main.py` đã đáp ứng toàn bộ đề gốc.

Đọc dữ liệu và cách chương trình chạy

1. **`import pandas as pd`** nạp thư viện Pandas. `pd.read_csv("students.csv")` đọc một file bảng từ **thư mục chạy hiện tại**. Nếu file không có ở đó, lệnh đọc phát sinh `FileNotFoundError`.
2. **`inputHelper(data)`** hiển thị lời nhắc rồi dùng `split(";")` để tách nhiều giá trị. Một mục nhập trống trả về `None`; các hàm tìm kiếm và tính điểm dùng điều này để kết thúc.
3. **`displayAll()`** đọc bảng, lấy cột tên rồi in từng học viên bằng `for` và số thứ tự.
4. **`studentSearch()`** lọc bảng bằng biểu thức Boolean `df["student_id"] == inputData`; nếu lọc ra hàng thì in, nếu không thì báo không tìm thấy.

5. **`studentAverage()`** lấy điểm Python, Math, ML của một học viên, tính **trung bình = tổng ba điểm chia 3**, dùng `round(..., 2)` để hiển thị hai chữ số, `max(...)` / `min(...)` để lấy điểm cao/thấp nhất.
6. **`analyzeSubject()`** gọi `.mean()`, `.sum()`, `.max()`, `.min()` trên từng cột điểm; `pd.DataFrame(data, index=...)` trình bày các thống kê theo môn.
7. **`analyzeClass()`** gọi `groupby("class_name")` để chia theo lớp, `.count()` để đếm sinh viên, `.mean()` tính trung bình; `pd.merge(..., on="class_name", how="outer")` ghép bảng mà vẫn giữ các nhóm lớp, `reset_index(inplace=True)` chuyển chỉ số thành cột, `rename(columns=..., inplace=True)` đổi tên cột, `round(2)` làm tròn.
8. **`top3()`** thêm cột `Average`, gọi `sort_values(by="Average", ascending=False)` để xếp cao xuống thấp; `drop(columns=..., inplace=True)` bớt cột thừa và `head(3)` lấy ba hàng đầu.
9. **`menu()`** dùng `while True`, `if/elif` và `break` để điều hướng 7 lựa chọn.

Những điểm phải biết khi tự chạy và đối chiếu đề

Tên cột `" name"` có dấu cách: CSV gốc trong PythoClass có header `student_id, name,class_name,Python,Math,ML`, nghĩa là tên cột thực tế là `" name"` có khoảng trắng đầu. Bởi vậy `df[" name"]` trong `displayAll()` chạy với **CSV gốc**, nhưng sẽ lỗi `KeyError` nếu dùng dữ liệu mẫu trong đề Word đã chuẩn hóa thành `"name"` không có dấu cách. Khi thay CSV, cần nhất quán tên cột.

`inputHelper()` có hạn chế: `input().split(";")` luôn trả các chuỗi `str`. Cách kiểm tra `type(blocks[i]).__name__` không ép sang số thực/nguyên, và nhánh báo sai số trường không dừng trước vòng lặp: nếu thiếu trường có thể xảy ra `IndexError`. Nên kiểm tra số trường rồi `return` hoặc báo lỗi, sau đó chuyển kiểu thực sự bằng `int()/float()` có xử lý `ValueError` khi mở rộng chức năng.

Gọi đệ quy để hỏi lại: `studentSearch()` và `studentAverage()` gọi lại chính nó sau mỗi lượt tra cứu. Cách này có thể dùng nhanh nhưng gọi nhiều lần có nguy cơ `RecursionError`; vòng `while` là lựa chọn an toàn hơn.

Đối chiếu với đề Word: `main.py` **không có** thao tác thêm sinh viên, kiểm tra mã trùng khi thêm, lưu sinh viên ra CSV và kiểm tra điểm 0–100. Do đó, nếu bài thi yêu cầu đủ các chức năng của Word, vẫn phải bổ sung chúng. Phiên bản `labs/pythoclass/LAB-10-file-csv/SOLUTION.py` là **lời giải tham khảo khác** đã triển khai các yêu cầu đó; không nên nhầm nó với `main.py` của bạn.

Bảo toàn nguồn: Phần code in phía trên giữ nguyên **các dòng lệnh thực thi** trong `PythoClass/main.py`; chỉ bổ sung chú thích tiếng Việt cuối dòng để ôn tập. Những vấn đề được phân tích ở đây thay vì âm thầm sửa logic chương trình. Dùng `students.csv` gốc cùng thư mục chạy khi muốn kiểm tra đúng hành vi mã nguồn.

Đi theo một lượt chạy thực tế từ đầu tới cuối

Khởi động: chương trình bắt đầu khi gọi `menu()` ở cuối file. `while True` tiếp tục in 7 lựa chọn; `input(...)` luôn trả về chuỗi, `.isnumeric()` kiểm tra xem có thể chuyển sang `int` trước khi đối chiếu `choice`. Lựa chọn 7 dùng `break` để dừng menu.

Dữ liệu Pandas: `pd.read_csv("students.csv")` đọc CSV thành `DataFrame`. Mỗi lần chọn một chức năng, chương trình lại đọc file: cách này đơn giản nhưng có chi phí đọc lại, và phụ thuộc thư mục chạy hiện tại. `df[df["student_id"] == inputData]` tạo mặt nạ Boolean rồi lọc hàng; khác với `df["student_id"]` vốn chỉ chọn một cột.

Các cấp thống kê: Khi xem *một học viên*, `result.values[0]` lấy hàng đầu của bảng lọc; các chỉ số 3, 4, 5 lần lượt là điểm Python, Math, ML theo thứ tự cột của CSV gốc. Khi xem *một môn*, `.mean()`, `.sum()`, `.max()`, `.min()` tổng hợp toàn bộ sinh viên. Khi xem *một lớp*, `groupby("class_name")` tạo các nhóm; `pd.merge(..., on="class_name", how="outer")` ghép bảng đếm người và bảng điểm trung bình. Cuối cùng, xếp hạng top 3 cần tính `Average`, sắp giảm dần rồi `head(3)`.

Lỗi và giới hạn quan trọng: `" name"` (có khoảng trắng) khác `"name"`; file CSV dùng sai header sẽ gây `KeyError`. `split(";")` trả về các chuỗi dù người dùng gõ số, nên so sánh tên kiểu với `formats` chưa kiểm tra dữ liệu thực sự. Nếu nhập thiếu mục, cách kiểm tra hiện tại có thể dẫn đến `IndexError`. Hai hàm tìm kiếm và xem điểm gọi lại chính nó nên nếu thao tác quá nhiều có thể tăng độ sâu đệ quy.

So với yêu cầu giảng viên: code hiện tại **chưa thêm học viên, chưa lưu CSV, chưa chặn trùng mã khi thêm và chưa xác nhận điểm 0–100**. Những chức năng này cần viết bổ sung nếu bài thi yêu cầu đúng nguyên văn đề Word. Bản in chỉ phân tích và chú thích `main.py` thật, không biến nó thành một lời giải khác rồi ghi nhầm nguồn.

5. Giải thích các định nghĩa

Tra cứu các thuật ngữ và hàm trong `main.py` theo ví dụ đơn giản bên dưới.

CSV (bảng phân tách bằng dấu phẩy)

Là gì, dùng khi nào? File gồm nhiều hàng và cột. Mỗi hàng trong Lab 10 là một học viên, mỗi cột ghi mã, tên, lớp hoặc điểm.

Ví dụ: CSV gốc có tiêu đề `student_id, name,class_name,Python,Math,ML`; dấu cách trước `name` là một phần tên cột.

Pandas và DataFrame

Là gì, dùng khi nào? Pandas là thư viện xử lý bảng; DataFrame là dạng bảng có tên cột, giúp lọc và tính thống kê.

Ví dụ: `pd.read_csv("students.csv")` đọc CSV; `df["Math"]` lấy một cột điểm.

Boolean filtering (lọc theo điều kiện)

Là gì, dùng khi nào? So sánh từng giá trị của cột với điều kiện, tạo ra True/False, từ đó chỉ giữ các hàng đúng.

Ví dụ: `df[df["student_id"] == "S001"]` lấy hàng sinh viên có mã S001.

Average / highest / lowest (trung bình / cao / thấp)

Là gì, dùng khi nào? Trung bình bằng tổng ba điểm chia 3; `max` tìm số lớn nhất, `min` tìm số nhỏ nhất.

Ví dụ: Điểm 80,70,90: trung bình 80, cao nhất 90, thấp nhất 70.

`groupby()` (gom theo nhóm)

Là gì, dùng khi nào? Gom sinh viên theo lớp rồi tính thống kê cho từng lớp thay vì cho cả trường.

Ví dụ: `groupby("class_name")["Math"].mean()` cho điểm Math trung bình mỗi lớp.

`pd.merge()` (ghép bảng)

Là gì, dùng khi nào? Gộp các bảng thống kê theo một cột chung; `on` xác định cột nối, `how="outer"` giữ khóa xuất hiện ở ít nhất một bảng.

Ví dụ: Ghép bảng đếm sinh viên với bảng điểm trung bình cùng khóa lớp.

`sort_values()` và `head()` (xếp hạng)

Là gì, dùng khi nào? `sort_values(...,ascending=False)` đưa điểm cao lên đầu, `head(3)` lấy ba hàng đầu.

Ví dụ: Sau khi tạo cột Average, kết quả đầu là ba sinh viên có trung bình cao nhất.

`split()` và kiểm tra đầu vào

Là gì, dùng khi nào? `split(";")` tách chuỗi tại dấu chấm phẩy nhưng vẫn trả về chuỗi, chưa chuyển sang số.

Ví dụ: Chuỗi `S001;8.5` tạo `S001` và `8.5`; muốn tính toán phải chuyển phần điểm qua `float()`.

Recursion / loop (đệ quy / vòng lặp)

Là gì, dùng khi nào? Đệ quy là hàm tự gọi lại chính nó; vòng lặp làm lại một nhóm lệnh mà không tạo thêm lần gọi hàm.

Ví dụ: Tìm sinh viên liên tiếp rất nhiều lần bằng đệ quy có nguy cơ RecursionError.

KeyError và FileNotFoundError

Là gì, dùng khi nào? KeyError xảy ra khi tên cột/khóa không tồn tại; FileNotFoundError xảy ra khi thiếu tệp cần đọc.

Ví dụ: Code đọc `df[" name"]` sẽ lỗi nếu CSV chỉ có cột `name` không có dấu cách.